OpenHydroQual

Settings Reference

October 19, 2026

Using this manual

Every field of the settings dialog has its own entry below, titled with the label shown on screen, so the table of contents can be used as an index. Each entry gives the default value, the available choices where a field offers them, and the name the field carries inside a saved model file.

Fields are grouped as the interface groups them. Most models need only the General settings; the Solver settings matter when a run misbehaves, and the Optimization and MCMC groups only when calibrating against measurements.

Contents

1 General settings **4**

1.1 Simulation start time 4

1.2 Simulation end time 4

1.3 Output File Name (All outputs) 4

1.4 Output corresponding to observed data 4

2 Solver settings **4**

2.1 Crank-Nicholson Time Weight 5

2.2 Number of threads to be used 6

2.3 Newton-Raphson convergence tolerance (relative) 6

2.4 Newton-Raphson per-block mass-balance tolerance 6

2.5 Newton iteration count below which the time step grows 6

2.6 Newton iteration count above which the time step shrinks 7

2.7 Newton-Raphson Time-step reduction factor 7

2.8 Newton-Raphson Time-step reduction factor when fail 7

2.9 Minimum time-step . . . 7
2.10 Initial time-step . . . 7
2.11 Maximum simulation time allowed (seconds) . . . 7
2.12 Maximum number of jacobian calculations allowed . . . 7
2.13 Rate output interpolation . . . 8
2.14 Make time series uniform . . . 8
2.15 Write Solution Details . . . 8
2.16 Linear solver for the Newton step . . . 8
2.17 Reassemble the Jacobian at every Newton iteration . . . 8
2.18 Reassemble the Jacobian when the time step changes by more than this factor . . . 9
2.19 Time series that limit the time step . . . 9
2.20 Write the outputs intermittently . . . 9
2.21 Record all block/link outputs . . . 9
2.22 Detect and suppress solution oscillation . . . 9
2.23 Oscillation detection tolerance . . . 10
2.24 Output write interval . . . 10
2.25 Maximum allowable time-step increase factor with respect to the initial time-step . 10
2.26 Maximum allowable time-step decrease factor with respect to the initial time-step 10
2.27 Transport Jacobian assembly . . . 10
2.28 Verify Jacobian assembly (diagnostic) . . . 10
2.29 Maximum oscillation interventions . . . 10
2.30 Response to detected oscillation . . . 11
2.31 Clean steps before relaxing the time-step ceiling . . . 11
2.32 Time steps between restore points . . . 11
2.33 Maximum reuses of one restore point . . . 11

3 Optimization **11**

3.1 Genetic Algorithm Population . . . 12
3.2 Number of generations . . . 12
3.3 Number of threads to be used . . . 12
3.4 Cross over probability . . . 12
3.5 Mutation Probability . . . 12
3.6 Shaking Scale . . . 13
3.7 Shake Scale Reduction Factor . . . 13
3.8 GA Output File Name . . . 13

3.9 Continue optimization using pre-created GA output file 13

4 MCMC **13**

4.1 Number of MCMC samples 14
4.2 Number of chains 14
4.3 Number of samples to be discarded (burn out) 14
4.4 Initial perturbation factor 14
4.5 The interval at which the samples are saved 14
4.6 Initially purturb the samples 15
4.7 Perform global sensitivity 15
4.8 Continue MCMC using pre-created file 15
4.9 Number of post-estimate realizations 15
4.10 Add error noise to the realizations 15
4.11 Number of threads to be used 15
4.12 Desired acceptance rate 15
4.13 Coefficient of change of purtubation factor 16
4.14 Covariance-adapted proposal 16
4.15 Covariance refresh interval 16
4.16 Samples before first covariance 16
4.17 Covariance proposal scale 16
4.18 Step-size adaptation interval 16
4.19 Covariance centering 16
4.20 MCMC Output file 17

1 General settings

These fields define what period is simulated and where the results are written. Most models need nothing beyond this group.

Field	Default
Simulation start time	`0`
Simulation end time	`1`
Output File Name (All outputs)	`output.txt`
Output corresponding to observed data	`observedoutput.txt`

Simulation start time

Default: `0` • **Name in file:** `simulation_start_time`

Start of the simulated period, in the time unit used by the model's input series.

Simulation end time

Default: `1` • **Name in file:** `simulation_end_time`

End of the simulated period. It must be long enough for the behaviour being studied to appear: a run that stops before breakthrough gives no answer rather than a partial one.

Output File Name (All outputs)

Default: `output.txt` • **Name in file:** `alloutputfile`

File receiving the recorded results for every block and link. This is the complete record and can be very large. Leave empty to suppress it entirely.

Output corresponding to observed data

Default: `observedoutput.txt` • **Name in file:** `observed_outputfile`

File receiving only the quantities declared as Observations, i.e. the modelled values at the points where measurements exist. Much smaller than the full output and the file to use when comparing against data.

2 Solver settings

These control how the equations are solved. The defaults are appropriate for ordinary problems; the fields below explain when a problem is not ordinary and which one to reach for.

Field	Default
Crank-Nicholson Time Weight	`1`
Number of threads to be used	`8`
Newton-Raphson convergence tolerance (relative)	`0.001`
Newton-Raphson per-block mass-balance tolerance	`0.001`
Newton iteration count below which the time step grows	`20`
Newton iteration count above which the time step shrinks	`40`
Newton-Raphson Time-step reduction factor	`0.75`
Newton-Raphson Time-step reduction factor when fail	`0.2`
Minimum time-step	`1e-6`
Initial time-step	`1e-2`
Maximum simulation time allowed (seconds)	`86400`
Maximum number of jacobian calculations allowed	`200000`
Rate output interpolation	`midpoint`
Make time series uniform	`Yes`
Write Solution Details	`No`
Linear solver for the Newton step	`Direct`
Reassemble the Jacobian at every Newton iteration	`No`
Reassemble the Jacobian when the time step changes by more than this factor	`2`
Time series that limit the time step	`Precipitation only`
Write the outputs intermittently	`No`
Record all block/link outputs	`Yes`
Detect and suppress solution oscillation	`No`
Oscillation detection tolerance	`0.01`
Output write interval	`100`
Maximum allowable time-step increase factor with respect to the initial time-step	`50`
Maximum allowable time-step decrease factor with respect to the initial time-step	`100000`
Transport Jacobian assembly	`Dependency`
Verify Jacobian assembly (diagnostic)	`0`
Maximum oscillation interventions	`4`
Response to detected oscillation	`rewind`
Clean steps before relaxing the time-step ceiling	`40`
Time steps between restore points	`200`
Maximum reuses of one restore point	`2`

Crank-Nicholson Time Weight

Default: `1` • **Name in file:** `c_n_weight`

Time-weighting factor for the Crank-Nicholson scheme (0.5 for implicit midpoint rule, 1 for fully implicit).

Number of threads to be used

Default: `8` • **Name in file:** `n_threads`

Number of processor threads to use for parallel computation during simulation.

Newton-Raphson convergence tolerance (relative)

Default: `0.001` • **Name in file:** `nr_tolerance`

Convergence criterion for the Newton iteration at each time step. It is a RELATIVE measure, not an absolute residual: the iteration stops once the residual has fallen to this fraction of its value at the start of the step, so 0.001 means the residual is reduced by three orders of magnitude and no further. Two other conditions also end the iteration – an absolute residual floor of 1e-12, and an update that has become negligible against the solution norm. Because the criterion is relative and per step, the error left behind accumulates over a long run: on a model whose outputs are compared against measurements of similar magnitude, the default can leave a numerical error that is an appreciable fraction of the model-measurement residual, and it is worth tightening until the results stop changing. A smaller value does not necessarily cost time. A better converged step is a more stable one, so the solver can often take larger steps afterwards and finish sooner; measure rather than assume.

Newton-Raphson per-block mass-balance tolerance

Default: `0.001` • **Name in file:** `nr_block_tolerance`

A second convergence test for the Newton iteration, applied to every block on its own. nr_tolerance and the update test are norms over the whole model, so they are set by its largest blocks: when some blocks are very large (a fixed-head boundary given a nominal storage of 1e12, for example) a small, stiff block can be accepted far out of balance. With this test a step is accepted only once each block's residual is below this fraction of its own throughput (storage change plus all its inflows and outflows), or the block has stopped changing. 0.001 bounds the mass-balance error of every block at about 0.1% of what flows through it; blocks whose throughput is below a thousandth of the largest are held to that level instead. The transport (constituent) solve applies the same test to each constituent in each block. 0 switches the test off (the behaviour before September 2026).

Newton iteration count below which the time step grows

Default: `20` • **Name in file:** `nr_niteration_lower`

After an accepted step the solver compares the largest Newton iteration count over all state variables with this value; below it the base time step is multiplied by 1/nr_timestep_reduction_factor. Raise it for models whose Newton legitimately needs many iterations (the Jacobian is assembled once per step and reused, so a stiff transport phase can need 20-30 chord iterations) – otherwise the step is throttled for no accuracy benefit.

Newton iteration count above which the time step shrinks

Default: `40` • **Name in file:** `nr_niteration_upper`

Counterpart to nr_niteration_lower: above this the base time step is multiplied by nr_timestep_reduction_factor. Keep it comfortably above nr_niteration_lower.

Newton-Raphson Time-step reduction factor

Default: `0.75` • **Name in file:** `nr_timestep_reduction_factor`

Factor to reduce the time-step when convergence slows down. Typical values range between 0.5 and 0.9.

Newton-Raphson Time-step reduction factor when fail

Default: `0.2` • **Name in file:** `nr_timestep_reduction_factor_fail`

Time-step reduction factor applied when Newton-Raphson iteration fails to converge.

Minimum time-step

Default: `1e-6` • **Name in file:** `minimum_timestep`

Absolute floor on the time step, regardless of the other limits.

Initial time-step

Default: `1e-2` • **Name in file:** `initial_time_step`

The step the run begins with, and the basis of the upper bound: the step can never exceed this value times the maximum increase factor. A very small value here therefore pins the step for the whole run unless that factor is raised to match. It is also the spacing on which results are written, so a ten times finer run produces ten times more output.

Maximum simulation time allowed (seconds)

Default: `86400` • **Name in file:** `maximum_time_allowed`

Real-world clock limit in seconds. When it expires the run stops wherever it has reached, so an unattended run cannot continue indefinitely.

Maximum number of jacobian calculations allowed

Default: `200000` • **Name in file:** `maximum_number_of_matrix_inverstions`

Overall budget of linear solves for the run.

Rate output interpolation

Default: `midpoint` • **Choices:** `step, midpoint` • **Name in file:** `rate_output_interpolation`

How flows (and other rates on links and sources) are put onto the output grid of the initial time step at the end of a run, before they are plotted or compared with observations. The solver's flow for a time step is the average over that step. 'midpoint' (default) places it at the middle of the step and interpolates linearly between steps, which gives a continuous hydrograph. 'step' (the behaviour before October 2026) holds it constant over the step, so long steps give a staircase. States (storage, depth, head) are always interpolated linearly.

Make time series uniform

Default: `Yes` • **Choices:** `Yes, No` • **Name in file:** `make_timeseries_uniform`

Resamples every time series onto a uniform grid whose spacing is the initial time step, so lookups index by arithmetic instead of scanning. Leave on unless a model carries long or finely-resolved time series: the number of points created is the record length divided by the initial time step, so a small initial time step can expand a short file into millions of points and the run will appear to hang before the first step. The resampling also smears steep edges, which adds a little volume. Set to No to use each series exactly as supplied.

Write Solution Details

Default: `No` • **Choices:** `Yes, No` • **Name in file:** `write_solution_details`

Produces a log of what the solver did: where it changed the time step, where it struggled and why. The first thing to switch on when a run behaves unexpectedly.

Linear solver for the Newton step

Default: `Direct` • **Choices:** `Direct, Inverse Jacobian, Sparse` • **Name in file:** `jacobian_method`

What is done with the assembled matrix. Direct factorises and solves, the normal choice. Inverse Jacobian forms the inverse explicitly: more work and slightly less accurate, but reusable across steps. Sparse stores only non-zero entries and pays off for large models where most of the matrix is empty.

Reassemble the Jacobian at every Newton iteration

Default: `No` • **Choices:** `Yes, No` • **Name in file:** `update_jacobian_every_iteration`

Off (the default) uses the classic chord scheme: the Jacobian is assembled once at the start of a step and reused, refreshed only when the step stops improving. That is cheap per iteration but converges linearly, so a stiff phase can need 20-30 iterations. On, the Jacobian is rebuilt every iteration, giving true Newton convergence – typically 1-2 iterations on systems that are linear in the state. Because the iteration count also drives the time-step adaptation, turning this on usually raises the time step as well.

Reassemble the Jacobian when the time step changes by more than this factor

Default: `2` • **Name in file:** `jacobian_dt_refresh_factor`

The reused (chord) Jacobian contains 1/dt, so it stops being a good approximation once the time step moves away from the step it was assembled with. When the time step is clamped to rapidly changing time series it changes almost every step, and a Jacobian kept for 50 steps can cost 20-30 Newton iterations per step. With this set, the Jacobian is reassembled whenever the applied time step differs from the one it was built with by more than this factor, up or down. 2 is the default; 1 or less switches the check off (reassemble only every 50 steps or after a failed step, the behaviour before September 2026).

Time series that limit the time step

Default: `Precipitation only` • **Choices:** `Precipitation only, All time series` • **Name in file:** `timestep_clamp_series`

A step is never allowed to run past a point where a time series starts to change. All time series applies this to every loaded series – precipitation, gate openings, imposed heads, inflows and weather inputs – so forcing that changes quickly is followed closely, at the cost of more steps. Precipitation only (the default) applies it to precipitation series alone: fewer and larger steps, but other forcing is only sampled at the step ends and short events such as a 15-minute gate operation can be stepped over.

Write the outputs intermittently

Default: `No` • **Choices:** `Yes, No` • **Name in file:** `write_intermittently`

Whether results are written during the run rather than only at the end. Writing during the run lets you watch progress, at the cost of rewriting the file each time.

Record all block/link outputs

Default: `Yes` • **Choices:** `Yes, No` • **Name in file:** `record_results`

Whether the values of every block and link are kept. Set to No, only quantities declared as Observations are stored. Because results are held in memory until the run ends this governs memory as well as file size: for a long run the difference can be several gigabytes against a few hundred kilobytes.

Detect and suppress solution oscillation

Default: `No` • **Choices:** `Yes, No` • **Name in file:** `oscillation_control`

Watches for a solution that oscillates, i.e. rises and falls on alternate steps rather than advancing. This can happen when the step is coarse relative to the fastest reaction, and nothing is reported when it does: each step converges, the run completes, and the results may contain excursions that cannot be real. Off by default because some models genuinely oscillate and only the modeller can say whether a reversal is physical.

Oscillation detection tolerance

Default: `0.01` • **Name in file:** `oscillation_tolerance`

A variable is flagged when it reverses direction twice in succession AND the movement exceeds this fraction of the variable's own magnitude. Both conditions are needed: the reversal pattern alone is satisfied by round-off, the magnitude alone by any rapidly changing but healthy solution.

Output write interval

Default: `100` • **Name in file:** `write_interval`

How often accumulated results are written to disk.

Maximum allowable time-step increase factor with respect to the initial time-step

Default: `50` • **Name in file:** `max_timestep_increase_factor`

A multiplier on the initial time-step that defines the largest step the run may ever take. It is a ceiling, not a rate of growth.

Maximum allowable time-step decrease factor with respect to the initial time-step

Default: `100000` • **Name in file:** `max_timestep_decrease_factor`

Bounds how far below the initial time-step the adaptive step may fall.

Transport Jacobian assembly

Default: `Dependency` • **Choices:** `Dependency, Full` • **Name in file:** `jacobian_assembly`

How the Jacobian is built. Full re-computes every equation for each perturbed unknown: always correct, but the work grows with the square of the model size. Dependency re-computes only the equations the perturbed unknown can reach, giving the same matrix far more quickly. Use Full only to check a suspected error in the dependency analysis.

Verify Jacobian assembly (diagnostic)

Default: `0` • **Name in file:** `verify_jacobian`

Assembles the Jacobian both ways and logs the largest disagreement. Roughly doubles the cost of every step; for diagnosis, not production.

Maximum oscillation interventions

Default: `4` • **Name in file:** `oscillation_max_reductions`

How many times the solver may intervene before giving up and reporting that the oscillation could not be removed, rather than refining indefinitely.

Response to detected oscillation

Default: `rewind` • **Choices:** `rewind, reduce` • **Name in file:** `oscillation_remedy`

Rewind returns to the last restore point, restarts at a fifth of the step, and discards results already written past that point, so the oscillation is removed from the output rather than merely stopped. Reduce keeps the current state and continues with a smaller step: much cheaper, but anything already written stays. Choose rewind when the recorded results matter most, reduce when run time does.

Clean steps before relaxing the time-step ceiling

Default: `40` • **Name in file:** `oscillation_relax_after`

Number of consecutive untroubled steps before the imposed ceiling is eased and the step allowed to grow again. Oscillation is often a passing episode, so holding the step down for the remainder of a long run can cost far more than the episode itself; but relaxing too eagerly invites the oscillation back. Set to 0 to hold the ceiling for the rest of the run.

Time steps between restore points

Default: `200` • **Name in file:** `restore_interval`

How often a complete snapshot of the model is saved. This controls both how much work is repeated when the solver goes back (it returns to the most recent snapshot) and how much time is spent taking them. Frequent snapshots make each rewind cheap but add overhead.

Maximum reuses of one restore point

Default: `2` • **Name in file:** `restore_point_max_uses`

How many times the solver may return to the same snapshot before refusing. The limit prevents returning to a state it cannot get past. If oscillation suppression reports that it gave up, raising this together with more frequent snapshots gives it more room.

3 Optimization

Used when the program is asked to find the parameter values that best reproduce a set of measurements. The method is a genetic algorithm: it keeps a collection of candidate parameter sets, scores each by running the whole model, and builds the next collection from those that scored best. These fields are ignored unless a calibration is run.

Field	Default
Genetic Algorithm Population	`40`
Number of generations	`40`
Number of threads to be used	`8`
Cross over probability	`1`
Mutation Probability	`0.02`
Shaking Scale	`0.05`
Shake Scale Reduction Factor	`0.75`
GA Output File Name	`GA_output.txt`
Continue optimization using pre-created GA output file	*(empty)*

Genetic Algorithm Population

Default: `40` • **Name in file:** `maxpop`

How many candidate parameter sets exist at a time. Population times generations is the number of model runs, which sets how long the calibration takes. A larger population searches more broadly per round and suits parameters that trade off against one another.

Number of generations

Default: `40` • **Name in file:** `ngen`

How many rounds of selection are run. More generations polish an answer that is already close.

Number of threads to be used

Default: `8` • **Name in file:** `numthreads`

How many candidates are solved at once. Nothing is gained from more threads than candidates, or than the machine has cores.

Cross over probability

Default: `1` • **Name in file:** `pcross`

Chance that two selected parents are combined rather than copied unchanged.

Mutation Probability

Default: `0.02` • **Name in file:** `pmute`

Chance that an individual parameter value is altered at random. Mutation lets the search escape a merely-good answer, so zero is unwise; far above a few percent turns the search into guesswork.

Shaking Scale

Default: `0.05` • **Name in file:** `shakescale`

Size of the random adjustment applied to parameter values, as a fraction of each parameter's range.

Shake Scale Reduction Factor

Default: `0.75` • **Name in file:** `shakescalered`

Multiplies the shaking scale after each generation, so the search is bold at first and increasingly fine as it settles.

GA Output File Name

Default: `GA_output.txt` • **Name in file:** `outputfile`

Records every candidate of every generation with its score: the file to open to see whether the search converged.

Continue optimization using pre-created GA output file

Default: *empty* • **Name in file:** `continue_based_on_filename`

Resumes a previous optimization from the last complete generation in its GA output file, instead of starting from a random population. The parameters recorded in the file must match the model's parameters exactly - same names, same order - or the run is refused. New generations are appended to the output file. Leave empty to start a new optimization.

4 MCMC

Optimization returns the single best answer; this returns the range of answers consistent with the data. The sampler wanders through parameter space, spending time in each region in proportion to how well it explains the measurements, so the spread of what it collects measures how well each parameter is actually constrained.

Field	Default
Number of MCMC samples	`1000`
Number of chains	`8`
Number of samples to be discarded (burn out)	`8`
Initial perturbation factor	`0.05`
The interval at which the samples are saved	`1`
Initially purturb the samples	`No`
Perform global sensitivity	`No`
Continue MCMC using pre-created file	*(empty)*
Number of post-estimate realizations	`10`

Field	Default
Add error noise to the realizations	`No`
Number of threads to be used	`1`
Desired acceptance rate	`0.15`
Coefficient of change of purtubation factor	`0.75`
Covariance-adapted proposal	`Yes`
Covariance refresh interval	`1000`
Samples before first covariance	`500`
Covariance proposal scale	`2.38`
Step-size adaptation interval	`50`
Covariance centering	`pooled`
MCMC Output file	`mcmc.txt`

Number of MCMC samples

Default: `1000` • **Name in file:** `number_of_samples`

How many steps each walker takes.

Number of chains

Default: `8` • **Name in file:** `number_of_chains`

How many independent walkers are run. Several walkers share the work, and agreement between walkers that started in different places is the usual evidence that the result can be trusted.

Number of samples to be discarded (burn out)

Default: `8` • **Name in file:** `number_of_burnout_samples`

Discards the beginning of each walk, which reflects where the walker started rather than the answer.

Initial perturbation factor

Default: `0.05` • **Name in file:** `initual_purturbation_factor`

Step size the sampler begins with.

The interval at which the samples are saved

Default: `1` • **Name in file:** `record_interval`

Keeps one sample in every so many. Consecutive samples resemble one another by the nature of the method, so this discards little information while keeping the file manageable.

Initially purturb the samples

Default: `No` • **Choices:** `Yes, No` • **Name in file:** `initial_purturbation`

Scatters the starting positions of the walkers, which is what makes agreement between them meaningful.

Perform global sensitivity

Default: `No` • **Choices:** `Yes, No` • **Name in file:** `perform_global_sensitivity`

Additionally reports how much each parameter contributes to the variation in the results.

Continue MCMC using pre-created file

Default: *empty* • **Name in file:** `continue_based_on_file_name`

Resumes a previous run from its samples instead of starting over: how a run is extended when it has not yet settled.

Number of post-estimate realizations

Default: `10` • **Name in file:** `number_of_post_estimate_realizations`

How many parameter sets are drawn from the result and re-run, so predictions come out as a range rather than a single line.

Add error noise to the realizations

Default: `No` • **Choices:** `Yes, No` • **Name in file:** `add_noise_to_realizations`

Whether the residual error is added to those predictions. Without it the range shows uncertainty in the parameters alone; with it, the uncertainty in an actual future measurement, which is wider and usually the more honest thing to report.

Number of threads to be used

Default: `1` • **Name in file:** `number_of_threads`

Spreads the chains over cores; more threads than chains achieves nothing.

Desired acceptance rate

Default: `0.15` • **Name in file:** `acceptance_rate`

Target fraction of proposed moves accepted. A step too large is nearly always rejected and one too small explores too slowly, so the sampler tunes its step size to hit this. Values between about 0.15 and 0.3 are usual.

Coefficient of change of purtubation factor

Default: `0.75` • **Name in file:** `purturbation_change_scale`

How quickly the step size is adjusted toward the desired acceptance rate.

Covariance-adapted proposal

Default: `Yes` • **Choices:** `Yes, No` • **Name in file:** `covariance_proposal`

Propose correlated steps drawn from the empirical covariance of the samples so far, instead of perturbing each parameter independently. Use this when parameters trade off against one another – a rate and a capacity that reproduce the same curve, for example. A diagonal proposal cannot move along such a ridge and the chains drift apart instead of mixing.

Covariance refresh interval

Default: `1000` • **Name in file:** `covariance_update_interval`

Samples between recomputing the proposal covariance. Only used when the covariance-adapted proposal is on.

Samples before first covariance

Default: `500` • **Name in file:** `covariance_min_samples`

How many post-burn-in samples to accumulate before the first covariance is formed. Until then the diagonal proposal is used.

Covariance proposal scale

Default: `2.38` • **Name in file:** `covariance_scale`

Step size multiplier, divided by sqrt(number of parameters). 2.38 is the optimal value for a Gaussian target (Roberts & Rosenthal 2001) and rarely needs changing.

Step-size adaptation interval

Default: `50` • **Name in file:** `adaptation_interval`

Steps (one sample of every chain) between adjustments of the step size toward the target acceptance rate. Smaller values tune the step faster, e.g. after a restart.

Covariance centering

Default: `pooled` • **Choices:** `pooled, chain` • **Name in file:** `covariance_centering`

pooled: the proposal covariance is taken about the mean of all samples. chain: about the mean of each chain (pooled within-chain covariance). Use chain while the chains have not mixed yet: the

pooled covariance is then dominated by the distances between the chains, the proposals become too long and the step size shrinks until the chains stall.

MCMC Output file

Default: `mcmc.txt` • **Name in file:** `samples_filename`

File receiving the recorded samples.